

CIBERSEGURIDAD · DEVSECOPS

Manual del Alumno

Sube tu proyecto y entiende el ciclo completo de seguridad automatizada (Shift-Left) en un pipeline CI/CD.

GitLab CI

Gitleaks

SonarQube

Trivy

OWASP ZAP

DefectDojo

Aprenderás a crear tu repositorio, generar tokens y API keys, configurar el pipeline y disparar todos los análisis de seguridad con un solo `git push`.

1. ¿Qué vas a construir y por qué?

DevSecOps significa integrar la **seguridad** dentro del proceso de desarrollo y entrega de software (Dev + Sec + Ops), de forma **automática**. La idea clave se llama «**Shift-Left**»: mover los controles de seguridad lo más «a la izquierda» posible del ciclo de vida — es decir, detectar los problemas **en el momento en que el desarrollador sube el código**, no semanas después en producción (cuando arreglarlos es caro y arriesgado).

En esta práctica montarás un **pipeline**: una secuencia automática de trabajos («jobs») que se ejecuta **sola** cada vez que haces `git push`. Cada job aplica una técnica de seguridad distinta y, al final, todos los hallazgos se centralizan en un gestor de vulnerabilidades.

1. Secrets Gitleaks credenciales en el código	2. SAST SonarQube análisis del código	3. SCA Trivy CVEs en librerías	4. Build Docker construye imagen	5. Container Trivy CVEs de la imagen	6. DAST OWASP ZAP ataca la app viva	7. Report DefectDojo consolida todo
--	--	---	---	---	--	--

El «wow» de la práctica: tú solo haces `git push`. Nadie ejecuta las herramientas a mano. El pipeline detecta tu código, corre las 7 etapas en cadena y deja un panel con todas las vulnerabilidades clasificadas. **Eso es DevSecOps.**

Las capas de seguridad (y qué cubre cada una)

Técnica	Herramienta	¿Qué busca?	Ejemplo que encuentra
Secret Scanning	Gitleaks	Credenciales escritas en el código o el historial de git	Una API key o password «hardcodeada»
SAST (estático)	SonarQube	Fallos analizando el código fuente , sin ejecutarlo	SQL Injection, uso de MD5, command injection
SCA	Trivy	Vulnerabilidades conocidas (CVEs) en las librerías que usas	Una versión de <code>pyyaml</code> con CVE
Container Scanning	Trivy	CVEs en el sistema operativo y paquetes de la imagen Docker	Paquete del SO base desactualizado
DAST (dinámico)	OWASP ZAP	Fallos atacando la aplicación en ejecución	XSS reflejado, cabeceras de seguridad ausentes
Vuln. Mgmt	DefectDojo	Consolida y gestiona todos los hallazgos anteriores	Panel único por severidad

Idea para recordar: ninguna herramienta sola es suficiente. Cada una mira una «capa» distinta (secretos, tu código, tus dependencias, la imagen, la app corriendo). Juntas dan una visión completa.

2. Las herramientas del laboratorio

El instructor ya tiene levantados estos servicios. Tú los usarás desde el navegador. Todas las contraseñas del lab son `DevSecOps2024!`.

Herramienta	Para qué sirve	URL (navegador)	Usuario
GitLab	Guarda tu código y ejecuta el pipeline (CI/CD)	<code>http://localhost:8929</code>	<code>root</code>
SonarQube	Servidor donde se publican los resultados del SAST	<code>http://localhost:9000</code>	<code>admin</code>
DefectDojo	Gestor central de vulnerabilidades	<code>http://localhost:8080</code>	<code>admin</code>
Juice Shop	App vulnerable que ZAP ataca (objetivo del DAST)	<code>http://localhost:3000</code>	—

Muy importante — dos tipos de URL. Tú abres las herramientas en `localhost:<puerto>`. Pero los **jobs del pipeline** corren dentro de contenedores y se comunican entre ellos por **nombre de contenedor**, no por `localhost`. Por eso en las variables usarás `http://devsecops-sonarqube:9000`, `http://devsecops-defectdojo:8081` y `http://devsecops-juiceshop:3000`.

3. El ciclo, paso a paso

Ahora construyes el flujo con tus propias manos. Sigue los pasos en orden.

1 Prepara el código de tu aplicación

Usarás una app web **intencionalmente vulnerable** (para que las herramientas tengan qué encontrar). Puedes partir de la carpeta `sample-project/` del laboratorio, que ya trae todo:

```
mkdir mi-app-devsecops
cp -r sample-project/* mi-app-devsecops/
cd mi-app-devsecops
ls
```

Archivos que verás y qué es cada uno:

- `app.py` — la aplicación (Flask) con vulnerabilidades plantadas: SQL injection, XSS, secretos hardcoded, hash MD5...
- `requirements.txt` — las dependencias, fijadas a versiones viejas con CVEs conocidos (para el SCA).
- `Dockerfile` — receta para construir la imagen de la app.
- `sonar-project.properties` — configuración del análisis de SonarQube.
- `.gitlab-ci.yml` — **el pipeline**. Es el corazón de todo (lo revisamos en el paso 6).

2 Crea tu proyecto en GitLab y genera tu token de acceso

2.1 Crear el proyecto (interfaz web):

1. Entra a **http://localhost:8929** (root / **DevSecOps2024!**).
2. Arriba a la derecha: **+** → **New project** → **Create blank project**.
3. Nombre: **mi-app-<tu-nombre>** . Visibilidad: **Private**. **Desmarca** «Initialize repository with a README». → **Create project**.

2.2 Generar un Personal Access Token (PAT): el envío de código por HTTPS necesita un token, no tu contraseña.

1. Avatar (arriba dcha.) → **Edit profile** → **Access Tokens**.
2. **Add new token**. Nombre: **push-token** . Marca los scopes **read_repository** y **write_repository** . → **Create**.
3. **Copia el token** (empieza por **glpat-**). No se vuelve a mostrar.

¿Qué es un token? Es una credencial temporal y con permisos limitados. En vez de usar tu contraseña (que da acceso a todo), generas un token que solo puede hacer lo que necesitas (aquí: subir código). Es una buena práctica de seguridad que verás repetirse con SonarQube y DefectDojo.

3 Sube tu código a GitLab

```
cd mi-app-devsecops
git init -b main
git add .
git config user.email "alumno@lab.local"
git config user.name "Alumno"
git commit -m "App vulnerable inicial"

# Pega TU token en la URL (reemplaza <TU-PAT>):
git remote add origin http://root:<TU-PAT>@localhost:8929/root/mi-app-tunombre.git
git push -u origin main
```

Refresca tu proyecto en GitLab: ya está tu código. **Aún no corre nada** — todavía falta darle al pipeline las llaves (tokens) de las herramientas. Eso es lo siguiente.

4 Genera el token de análisis de SonarQube

1. Entra a **http://localhost:9000** (admin / **DevSecOps2024!**).
2. Avatar «A» (arriba dcha.) → **My Account** → **Security**.
3. En **Generate Tokens**: Name = **token-ci** , Type = **User Token** → **Generate**.
4. **Copia el token** (empieza por **squ_**).

Este token permitirá que el job de SonarQube publique el análisis de tu código en el servidor.

5 Obtén la API key de DefectDojo

Puedes hacerlo por la interfaz o por API (practica la API, es más «DevSecOps»):

```
curl -s -X POST http://localhost:8080/api/v2/api-token-auth/ \
  --data-urlencode "username=admin" \
  --data-urlencode "password=DevSecOps2024!"
# Respuesta: {"token":"xxxxxxxxxxxxx..."} ← copia ese valor
```

Con esta API key, el último job del pipeline importará automáticamente todos los reportes a DefectDojo.

6 Conecta todo: variables CI/CD del proyecto

Aquí es donde el pipeline recibe las «llaves». En tu proyecto de GitLab: **Settings** → **CI/CD** → **Variables** → **Add variable**, y añade estas cinco:

Key (nombre)	Value (valor)	Masked
SONAR_HOST_URL	http://devsecops-sonarqube:9000	No
SONAR_TOKEN	el <code>squ_...</code> del paso 4	Sí
DEFECTDOJO_URL	http://devsecops-defectdojo:8081	No
DEFECTDOJO_API_KEY	la key del paso 5	Sí
DAST_TARGET	http://devsecops-juiceshop:3000	No

Buena práctica: los tokens (secretos) van aquí como variables **enmascaradas** (Masked), **nunca** escritos dentro del `.gitlab-ci.yml`. Así no acaban expuestos en tu repositorio. El pipeline los lee como `${SONAR_TOKEN}`, etc.

7 Entiende el pipeline (`.gitlab-ci.yml`)

Este archivo, en la raíz de tu repo, define las 7 etapas. Un fragmento clave:

```
stages: [secrets, sast, sca, build, container, dast, report]

secret_scanning:      # 1) busca secretos con Gitleaks
  stage: secrets
  image: ghcr.io/gitleaks/gitleaks:v8.18.4
  script:
    - gitleaks detect --source . --report-format json \
      --report-path gitleaks-report.json --verbose
  allow_failure: true  # si ENCUENTRA secretos, avisa pero no bloquea
```

Cosas que debes entender del pipeline:

- **Cada etapa usa una imagen distinta** (la herramienta empaquetada en Docker). GitLab la descarga y ejecuta el `script`.
- `allow_failure: true` hace que, si una herramienta encuentra problemas, el job se ponga «naranja» (aviso) pero el pipeline continúe. Así vemos **todos** los hallazgos de todas las etapas.
- El job de **ZAP** incluye `mkdir -p /zap/wrk` antes de escanear (si no, no genera el reporte).

- El job de **report** envía cada reporte a DefectDojo con `product_type_name` para que cree el producto solo.

8 Dispara el pipeline con un push

```
git add .gitlab-ci.yml
git commit -m "Anade pipeline DevSecOps"
git push
```

Ve a tu proyecto → **Build** → **Pipelines**. Verás las 7 etapas ejecutándose en cadena, **solo por haber hecho push**. Haz clic en cada job para leer su log en tiempo real.

¿Por qué **secret_scanning** sale «naranja/failed»? Porque **encontrar** secretos hace que Gitleaks devuelva error — y eso es **lo que queremos**. Con `allow_failure: true`, el pipeline sigue. No es un fallo del lab: es la herramienta haciendo su trabajo.

9 Interpreta los resultados (cierra el ciclo)

1. **SonarQube** (localhost:9000) → tu proyecto: vulnerabilidades del código (SQLi, MD5, command injection...).
2. **DefectDojo** (localhost:8080) → producto «**DevSecOps Lab**»: **todos** los hallazgos (Gitleaks + Trivy + ZAP) clasificados por severidad (Critical / High / Medium / Low / Info).
3. Compara: ¿qué encontró cada herramienta? ¿Qué capa cubre cada una? Ahí entiendes por qué se necesitan todas.

4. Reto final: de «aviso» a «bloqueo» (Quality Gate)

Un pipeline que solo avisa no impide desplegar código inseguro. Conviértelo en un control que **bloquee**:

1. En `.gitlab-ci.yml`, en el job `secret_scanning`, cambia `allow_failure: true` por `false`.
2. `git commit -am "quality gate" && git push`.
3. Observa: ahora, si hay secretos, el pipeline se pone **ROJO** y (con Merge Requests) **impediría fusionar** el código.

Extra: corrige una vulnerabilidad real (quita los secretos de `app.py` o parametriza la consulta SQL), vuelve a pushear y comprueba que el hallazgo **desaparece**. Acabas de vivir el ciclo completo: detectar → corregir → verificar.

5. Glosario rápido

Término	Significado
CI/CD	Integración/Entrega Continua: automatizar build, pruebas y despliegue en cada cambio.
Pipeline	Secuencia automática de jobs que se ejecuta ante un evento (aquí, un <code>git push</code>).
Runner	El «motor» que ejecuta los jobs del pipeline. Sin runner online, el pipeline queda en «pending».
Shift-Left	Adelantar la seguridad al inicio del ciclo (en el push), no al final (producción).
SAST / DAST	Análisis estático (del código) / dinámico (de la app corriendo).
SCA	Software Composition Analysis: analizar las dependencias/librerías de terceros.
CVE	Identificador público de una vulnerabilidad conocida (p. ej. CVE-2020-14343).
Quality Gate	Umbral que, si no se cumple, bloquea la entrega (falla el pipeline).

6. Si algo no funciona

- **El pipeline se queda en «pending»:** no hay runner disponible. Avisa al instructor (revisa **Admin** → **CI/CD** → **Runners**).
- **El job de SonarQube/DefectDojo falla con error de token:** revisa que copiaste bien las variables del paso 6 (sin espacios) y que las URLs internas son `devsecops-...`, no `localhost`.
- **No ves hallazgos en DefectDojo:** abre el job `report_defectdojo` y mira los códigos HTTP; deben ser 201.

¡Lo lograste! Montaste, de punta a punta, un pipeline que automatiza la seguridad de una aplicación. Este es exactamente el flujo que usan los equipos reales para no depender de revisiones manuales. Ahora la seguridad viaja **con** el código, en cada push.